# RLM Sample Knowledge Dataset

A small multi-section PDF for testing a Recursive Language Model pipeline

## Purpose

This document is intentionally structured like a compact technical report. It contains definitions, examples, numbers, comparisons, and a final case study. The RLM can import this PDF, split it into chunks, recursively analyze the chunks, and answer questions using evidence from the document.

## 1. Introduction to Artificial Intelligence

Artificial intelligence (AI) is the field of building computer systems that perform tasks that normally require human intelligence. Typical tasks include classification, prediction, language understanding, planning, recommendation, and decision support. AI systems may use rules, statistical models, machine learning, or deep neural networks.

Machine learning (ML) is a subset of AI in which a system learns patterns from data rather than relying only on manually written rules. A supervised learning system is trained from labeled examples, while an unsupervised system looks for structure without labels. Reinforcement learning learns through actions, rewards, and penalties.

A useful distinction is between a model and an application. A model maps inputs to outputs. An application adds data ingestion, business logic, tool use, storage, monitoring, and evaluation around the model. An RLM belongs mainly to the application and orchestration layer.

## 2. Machine Learning Workflow

A standard machine learning workflow is: define the problem, collect data, clean the data, split the data, train a model, evaluate it, and deploy it. A common split is training data for learning, validation data for model selection, and test data for final evaluation.

Example dataset: a support team has 1,000 messages. Of these, 600 are billing questions, 250 are technical questions, and 150 are account questions. If a classifier is evaluated on 200 held-out messages and correctly labels 176, its accuracy is 88%. Accuracy is useful when class sizes are reasonably balanced, but precision, recall, or F1-score may be more informative when they are not.

Data leakage occurs when information that should be unavailable during training or evaluation accidentally enters the model input. Leakage can produce unrealistically high test performance. A good evaluation pipeline keeps test information isolated until final measurement.

## 3. Neural Networks and Transformers

A neural network contains layers of parameters that transform an input into an output. During training, a loss function measures error, and an optimization algorithm updates parameters to reduce that loss. Deep networks contain many layers and can learn complex functions.

A transformer uses attention mechanisms to let each token incorporate information from other tokens. In self-attention, queries, keys, and values are combined to compute weighted representations. This makes the architecture effective for language and other sequence tasks.

For a language model, the input can be represented as tokens. The model predicts the next token based on the context available to it. During inference, a generated answer can be represented as a sequence of token predictions. The model itself does not automatically know whether a statement is supported by an external document; retrieval and evaluation components can provide that grounding.

## 4. Small Language Models

A small language model can be useful for local experiments because it requires fewer computational resources. The model may be less capable than a large frontier model, but it is sufficient to demonstrate prompting, classification, planning, tool selection, and recursive orchestration.

For an RLM experiment, the small model can play three roles: planner, worker, and evaluator. The planner proposes how to decompose a difficult task. A worker solves a subtask using a document chunk or a tool. The evaluator checks whether a proposed result is correct, relevant, and sufficiently supported.

A practical experiment can use a model in the roughly 1B to 3B parameter range. The exact model is less important than the controller design, because the controller determines when to recurse, what information to pass, how results are combined, and when an answer is rejected.

## 5. Retrieval and Document Processing

When an RLM receives a large document, it should not blindly pass the entire document to every model call. A document loader first extracts text. A chunker then divides the text into manageable pieces, often preserving headings so that context is not destroyed.

Example chunking strategy: target chunk size 500 words with 80 words of overlap. For a 4,000-word report, this produces approximately 10 chunks, depending on paragraph boundaries. Overlap helps preserve context when a sentence or explanation spans two chunks.

A retrieval step can select the most relevant chunks for a question. Simple lexical retrieval can use keyword overlap, while semantic retrieval can use embeddings. The RLM can also recursively inspect all chunks of a section when the task requires broad coverage instead of only retrieving a few similar passages.

## 6. Recursive Language Model Architecture

An RLM is an orchestration system in which a language model can solve a task by creating smaller tasks and invoking the same solving mechanism on those subtasks. The parent process keeps control over recursion depth, task count, intermediate results, and final synthesis.

Suppose the question is: “Compare the three learning approaches described in this report and recommend one for a small local project.” The RLM may create three subtasks: summarize supervised learning, summarize unsupervised learning, and summarize reinforcement learning. It can then create a comparison task using the three summaries and finally a recommendation task constrained by local compute.

A useful recursion trace is: request -> planner -> subtasks -> child RLM calls -> child results -> synthesis -> evaluator -> final response. If an evaluator rejects a result, the system can retry the failed subtask with more context or create a narrower subtask.

## 7. Evaluation Strategy

An RLM should be evaluated on more than final-answer accuracy. Useful metrics include answer correctness, evidence coverage, number of model calls, recursion depth, latency, token usage, estimated cost, and retry count.

For example, suppose three approaches are tested on the same 20 questions. System A answers 17 correctly using 20 model calls. System B answers 18 correctly using 42 calls. System C answers 19 correctly using 65 calls. The best choice depends on the goal: if accuracy is the priority, C may be preferable; if cost and speed matter, A or B could be better.

An evaluator can return a structured result such as correct=true, score=0.92, missing_evidence=false, and reason=“The numerical comparison is supported by sections 2 and 5.” Structured evaluations make automated retries and benchmarking easier.

## 8. Case Study: Local RLM for an Operations Report

A fictional operations team tracks 12 monthly metrics. Three metrics are service response time, ticket backlog, and first-contact resolution. In January the values are 18 minutes, 420 tickets, and 72%. In February they are 16 minutes, 390 tickets, and 75%. In March they are 14 minutes, 310 tickets, and 79%.

From January to March, response time decreases by 4 minutes, backlog decreases by 110 tickets, and first-contact resolution increases by 7 percentage points. The combined trend suggests improved operational performance. However, a responsible analysis should not claim causation without additional evidence about staffing, ticket mix, policy changes, or seasonality.

A document-based RLM could answer: “What changed from January to March, which metric improved the most, and what should be investigated next?” It would need to extract the monthly values, calculate changes, compare the three metrics, and separate observed facts from recommendations.

## 9. RLM Test Tasks

Task A - Fact extraction: What were the response-time values in January, February, and March?

Task B - Calculation: By how many tickets did backlog decrease from January to March?

Task C - Comparison: Which of the three case-study metrics shows the largest percentage-point improvement?

Task D - Multi-step reasoning: Summarize the operational trend and provide two investigation questions without claiming causation.

Task E - Broad document analysis: Compare supervised, unsupervised, and reinforcement learning using the definitions in sections 1 and 4, then recommend which is easiest to prototype locally and explain why.

| Task | Type | Primary operation |
|---|---|---|
| A | Extraction | Read 3 values |
| B | Calculation | Subtract 310 from 420 |
| C | Comparison | Compare 7-point improvement |
| D | Multi-step | Extract + interpret + recommend |
| E | Broad analysis | Compare sections + synthesize |

## 10. Expected RLM Behavior

For Task A, the RLM can retrieve or inspect the case-study chunk and answer directly. Task B can be solved using arithmetic after extracting the two relevant numbers. Task D may benefit from separate subtasks for factual changes, interpretation, and recommendation constraints. Task E should recurse because the evidence is distributed across multiple sections and requires a synthesis step.

The final system should expose a trace containing the root task, child tasks, retrieved chunks, intermediate answers, evaluator decisions, and final synthesis. This makes the RLM auditable and helps identify whether an error came from retrieval, reasoning, arithmetic, or synthesis.